TASK 1 — Dynamic Model of the 6-DOF UAV Quadcopter (FINAL)

HACKSIMUL8 2026 · Department of Mechanical Engineering, PES University · 2 September 2026
Reference: Mien, T. & Tu, T. (2024), *IJRCS* 4(4), 1712–1730, doi:10.31763/ijrcs.v4i4.1594

> **Task 1 as stated:** *"Develop the dynamic mathematical model of the 6 DOF UAV Quadcopter in MATLAB and SIMULINK in terms of state space equations."*

Status: COMPLETE — 25/25 verification checks passed.

---

1. What the task requires, decomposed

Three deliverables are contained in that one sentence:

1. a **dynamic model** — equations describing how the quadcopter moves
2. expressed in **state-space form** — $\dot{x} = Ax + Bu$
3. implemented in **MATLAB *and* Simulink** — both, not one

We add a fourth of our own: **evidence the model is correct**, because an unverified model is an assertion.

---

2. Physical setup

2.1 Six degrees of freedom, four actuators

	Freedoms	Symbol
Translation	along x, y, z	ξ = [X, Y, Z]
Rotation	about x, y, z	η = [φ, θ, ψ]

- **φ (roll)** — rotation about body x
- **θ (pitch)** — rotation about body y
- **ψ (yaw)** — rotation about body z

The quadcopter has **6 DOF but only 4 rotors**, so it is **underactuated**. It cannot translate sideways directly — it must **tilt** so that thrust acquires a horizontal component. This appears in the linear model as $\ddot{X} = g\cdot\theta$ and $\ddot{Y} = -g\cdot\phi$, and it is the single most important structural fact about the vehicle.

2.2 Two coordinate frames

- **Inertial (ground) frame** $O{-}xyz$ — fixed to earth; position and altitude measured here
- **Body frame** $O\_B{-}x\_B\ y\_B\ z\_B$ — fixed to the airframe at the centre of gravity; **thrust always acts along +z_B**, whatever the attitude

The entire model arises from reconciling these: forces are produced in the body frame, motion is measured in the inertial frame, and the rotation matrix links them.

2.3 The three stated assumptions

Taken verbatim from the problem statement, and each has a modelling consequence:

Assumption	Consequence
homogeneous, symmetric block	inertia tensor is **diagonal**, `I = diag(Ixx, Iyy, Izz)`
centre of quadcopter = centre of gravity	no offset terms in the torque balance
elasticity neglected, transmission rigid	no flexible-body dynamics

---

3. Parameters (Table 1 — given, not chosen)

Parameter	Symbol	Value
Quadcopter mass	m	0.516 kg
Arm length	l	0.225 m
Gravity	g	9.81 m/s^2
Inertia moment of the rotor	I_M	3.368×10^{-5} $kg\cdot m^2$
Thrust factor of rotor	k	2.996×10^{-6} $N\cdot s^2$
Drag coefficient	b	1.260×10^{-7} $N\cdot m\cdot s^2$
Inertial constants	Ixx, Iyy	4.984×10^{-3} $kg\cdot m^2$
	Izz	8.958×10^{-3} $kg\cdot m^2$

Cross-checked against the source paper's Table 1 — identical in all eight values.

Derived quantities

Quantity	Formula	Value
Hover weight	W = mg	**5.0620 N**
Hover rotor speed	ω = √(W/4k)	**649.9 rad/s** each (~6200 rpm)
Max thrust	U1_max = 4W	**20.248 N**
Max upward acceleration	U1_max/m − g	**29.43 m/s^2** (≈ 3 g)

A gap in the given data, and how we handled it

The source paper's Eq. (6) includes **air-resistance coefficients Ax, Ay, Az**:

```
m·ẍ + Ax·ẋ = F(CψSθCϕ + SψSϕ)
m·ÿ + Ay·ẏ = F(SψSθCϕ − CψSϕ)
m·z̈ + mg + Az·ż = F(CϕCθ)
```

Neither Table 1 in the problem statement nor Table 1 in the paper gives values for them. We therefore set `Ax = Ay = Az = 0`, which is the only consistent reading of the data supplied. The code retains the terms (`P.Ax, P.Ay, P.Az` in `quad_params.m`) so a non-zero value can be substituted if a judge asks.

This choice has a direct consequence in Task 2: with `Az = 0` the altitude channel is a *pure* double integrator, which is why the Ziegler–Nichols ultimate-gain method is degenerate for us but not for the reference paper.

---

4. Rotor forces to control inputs

4.1 Geometry (Fig. 1 — cross / "plus" configuration)

Rotor	Position	Spin sense
1	(l, 0, 0) — on +x_B	CW
2	(0, −l, 0) — on −y_B	CCW
3	(−l, 0, 0) — on −x_B	CW
4	(0, l, 0) — on +y_B	CCW

Rotors 1,3 spin opposite to 2,4, so in hover their reaction torques cancel and the vehicle does not spin about z.

4.2 Each rotor produces

- **thrust** `f_i = k·ω_i²` along +z_B
- **reaction torque** `τ_i = b·ω_i²` about z_B

Both scale with **ω²** — this is where the input nonlinearity enters.

4.3 Deriving the moments with τ = r × F

For rotor 4 at `r₄ = (0, l, 0)` with `F₄ = (0, 0, f₄)`:

```
τ₄ = r₄ × F₄ = |i  j  k |
               |0  l  0 |  =  (l·f₄, 0, 0)      → positive roll
               |0  0  f₄|
```

Repeating for all four rotors gives the **four virtual control inputs**:

```
U1 = k(ω1² + ω2² + ω3² + ω4²)      total thrust        [N]
U2 = k(ω4² − ω2²)                  roll  input  → τφ = l·U2
U3 = k(ω3² − ω1²)                  pitch input  → τθ = l·U3
U4 = b(ω1² + ω3² − ω2² − ω4²)      yaw torque          [N·m]
```

Why this matters: controllers are designed against `U1…U4` (thrust and three moments), which are physically intuitive and decoupled. Conversion back to individual rotor speeds happens only at the end.

Verified: all four rotors at ω_hover = 649.9 rad/s gives `U1 = 5.0620 N` (exactly the weight) with all three moments zero — confirming the mapping and the hover point together.

---

5. The nonlinear model

5.1 Translational dynamics

Thrust `F_b = [0, 0, U1]ᵀ` is rotated into the inertial frame by the ZYX Euler rotation matrix `R = Rz(ψ)Ry(θ)Rx(ϕ)`, then Newton's second law with gravity gives:

```
ẌẌ = (U1/m)(cos ϕ sin θ cos ψ + sin ϕ sin ψ)
ŸŸ = (U1/m)(cos ϕ sin θ sin ψ − sin ϕ cos ψ)
ŻŻ = −g + (U1/m)(cos ϕ cos θ)
```

The third equation drives all of Task 2. Vertical acceleration depends on thrust multiplied by `cos ϕ cos θ` — tilt the vehicle and it loses lift. At 17° of pitch, `cos(0.30) = 0.9553`, so **4.47% of lift disappears**.

5.2 Rotational dynamics — the paper's Euler–Lagrange form

We implement the reference paper's **Eq. (11)** directly:

```
η̈ = J(η)⁻¹ · [ τ_B − C(η, η̇)·η̇ ]
```

where `η = [ϕ θ ψ]ᵀ`, and:

`J(η)` — the configuration-dependent inertia matrix (their Eq. 8):

```
        ⎡ Ixx              0                    −Ixx·Sθ                          ⎤
J(η) =  ⎢ 0      Iyy·Cϕ² + Izz·Sϕ²      (Iyy−Izz)·Cϕ·Sϕ·Cθ                      ⎥
        ⎣ −Ixx·Sθ  (Iyy−Izz)·Cϕ·Sϕ·Cθ   Ixx·Sθ² + Iyy·Sϕ²·Cθ² + Izz·Cϕ²·Cθ²     ⎦
```

`C(η, η̇)` — the Coriolis matrix (their Eq. 12), containing the gyroscopic and centripetal terms. All nine elements are implemented verbatim in `quad_dynamics.m`.

`τ_B` — body torques (their Eq. 19):

```
τ_B = [ l·k(ω4² − ω2²),  l·k(ω3² − ω1²),  b(ω1² − ω2² + ω3² − ω4²) ]ᵀ
    = [ l·U2,            l·U3,            U4 ]ᵀ
```

Why this formulation rather than the simpler one. Many quadcopter papers use the simplified body-frame Euler equations:

```
φ̈ = ((Iyy−Izz)/Ixx)·θψ̇ − (I_M/Ixx)·θ̇Ω_r + (l/Ixx)·U2      ...etc
```

Those agree with Eq. (11) **only at hover**. We measured the divergence:

Tilt	Difference between the two formulations
0°	2.4e-03 rad/s²
17.2°	**4.85 rad/s² (≈22% relative)**
40°	21.4 rad/s² (≈47%)

Since the task is to reproduce the paper's model, we implement **Eq. (11)**. Agreement is **machine precision at every tilt angle** — see §8.

The simplified form remains available as `P.rot_model = 'euler'` for comparison; `'lagrange'` is the default.

> **One honest note:** the paper's Eq. (11) contains **no rotor gyroscopic term**, even though `I_M` appears in its Table 1. Our `'lagrange'` model follows the paper exactly and therefore omits it too. The `'euler'` model retains it. If asked why `I_M` is tabulated but unused: that is the paper's choice, and we matched it.

Note `Izz ≈ 2 × Ixx`. The vehicle is much harder to rotate about the vertical axis, which is why **yaw is always the slowest channel**.

5.3 The Euler-rate formulation, and its one singularity

The Euler–Lagrange form works directly in **Euler rates** ($\dot{\varphi}$, $\dot{\theta}$, $\dot{\psi}$) rather than body rates (p, q, r), which is exactly what the paper does — the matrix `J(η)` already absorbs the transformation `J = Wη ᵀ I Wη`, where

$$W_\eta = \begin{bmatrix} 1 & 0 & -\sin\theta \\ 0 & \cos\phi & \cos\theta\sin\phi \\ 0 & -\sin\phi & \cos\theta\cos\phi \end{bmatrix}$$

So there is **no small-angle approximation** in our rotational dynamics — that was the point of adopting Eq. (11).

The one caveat: `J(η)` becomes singular at θ = ±90°, which is gimbal lock of the ZYX Euler parameterisation, not a modelling error. The implementation guards against it by falling back to the hover-diagonal inertia — well outside any flight envelope, but it means the model never returns `Inf`.

---

6. State-space form

6.1 State vector (12 states)

Two states per degree of freedom, since each obeys a second-order ODE:

```
x = [ X  Ẋ  Y  Ẏ  Z  Ż  ϕ  ϕ̇  θ  θ̇  ψ  ψ̇ ]ᵀ
u = [ δU1  U2  U3  U4 ]ᵀ
```

6.2 Linearisation about hover

Equilibrium: all angles zero, all rates zero, `U1 = W = 5.0620 N`. Applying `sin ϕ ≈ ϕ`, `cos ϕ ≈ 1` and dropping products of small terms:

Nonlinear	Linearised
`Ẍ = (U1/m)(cϕsθcψ + sϕsψ)`	`Ẍ = g·θ`
`Ÿ = (U1/m)(cϕsθsψ − sϕcψ)`	`Ÿ = −g·ϕ`
`Z̈ = −g + (U1/m)cϕcθ`	`Z̈ = δU1/m`
`ϕ̈ = … + (l/Ixx)U2`	`ϕ̈ = (l/Ixx)·U2`
`θ̈ = … + (l/Iyy)U3`	`θ̈ = (l/Iyy)·U3`
`ψ̈ = … + (1/Izz)U4`	`ψ̈ = (1/Izz)·U4`

6.3 The A and B matrices

Non-zero entries only:

```
A(1,2)  = 1          A(2,9)  = +9.8100      (Ẍ = +g·θ)
A(3,4)  = 1          A(4,7)  = −9.8100      (Ÿ = −g·ϕ)
A(5,6)  = 1          B(6,1)  =  1.9380      (= 1/m)
A(7,8)  = 1          B(8,2)  = 45.1445      (= l/Ixx)
A(9,10) = 1          B(10,3) = 45.1445      (= l/Iyy)
A(11,12)= 1          B(12,4) = 111.6321     (= 1/Izz)
```

`C = I₁₂`, `D = 0`.

`A(2,9)` and `A(4,7)` are underactuation made visible. They say the only way to accelerate horizontally is to tilt — there is no `B` entry that translates directly.

6.4 Four decoupled channels

Channel	Equation	Transfer function	Numeric
Altitude	`Z̈ = δU1/m`	`1/(m s²)`	`1/(0.516 s²)`
Roll	`ϕ̈ = (l/Ixx)U2`	`l/(Ixx s²)`	`45.14/s²`
Pitch	`θ̈ = (l/Iyy)U3`	`l/(Iyy s²)`	`45.14/s²`
Yaw	`ψ̈ = (1/Izz)U4`	`1/(Izz s²)`	`111.63/s²`

Every channel is a double integrator. This is the most useful structural fact in the whole problem and drives every decision in Task 2.

---

7. Why "double integrator" matters

`G(s) = 1/(m s²)` has **two poles at the origin**:

1. **Marginally stable, not stable.** Constant force → output grows as t^2. Verified: a 2% thrust error diverges quadratically.
2. **Proportional control alone cannot stabilise it.** `m·s² + Kp = 0` gives poles at `±j√(Kp/m)` — sustained oscillation at *every* gain. **Derivative action is mandatory.**
3. **The ZN ultimate-gain method is degenerate** — no unique Ku exists. (See TASK2_final §3 for the important qualification regarding the reference paper.)

---

8. Verification — the evidence

Anyone can write equations; the mark is in proving them. All checks pass:

#	Check	Result	Interpretation
1	Table 1 parameters (9 values)	all exact	inputs correct
2	Hover residual ‖ẋ‖	**0.000e+00**	hover is an exact equilibrium
3	`max|A − A_numerical|`	**1.636e-12**	hand derivation = finite differences
4	`max|B − B_numerical|`	**3.122e-10**	same for B
5	Structural entries (6 checks)	all match	physics is in the right slots
6	`rank(ctrb(A,B))`	**12 of 12**	fully controllable
7	Open-loop poles	all 12 at origin	cannot fly without feedback
8	Rotor mapping at hover	U1 = 5.0620 N, moments 0	mapping and hover consistent
9	Roll/pitch sign convention	both positive as expected	no inverted axis

Check 3 is the one to put on a slide. It finite-differences the nonlinear model and compares against the hand-derived matrices entry by entry. Agreement to twelve decimal places converts "trust our algebra" into "here is the proof."

Run it yourself: `verify_all` in the `solution/` folder.

---

9. Files — what each one does

Core model files

File	Purpose
`quad_params.m`	Returns struct `P` with all 8 Table 1 parameters, derived quantities (W, ω_hover, U1_max), actuator limits, drag placeholders (`Ax,Ay,Az` = 0), and the 100 Hz controller sample time. **Every other file gets its numbers from here** — nothing is hard-coded.
`quad_dynamics.m`	The full nonlinear 6-DOF model. Input: state `x` (12×1), control `U` (U1–U4 plus Ω_r), params `P`, optional disturbance force. Output: $\dot{x}$ (12×1). Implements §5.1 and §5.2 exactly.
`rotor2U.m`	Maps four rotor speeds `[ω1 ω2 ω3 ω4]` to `[U1 U2 U3 U4]` and the residual speed Ω_r, using the τ = r × F geometry of §4. Sign conventions documented inline.

Task 1 scripts

File	Purpose
`task1_statespace.m`	The Task 1 deliverable. Builds A, B, C, D analytically; verifies by finite-differencing the nonlinear model; forms the four channel transfer functions; runs the controllability test; simulates open-loop divergence. Saves `task1_model.mat`.
`verify_all.m`	Independent re-verification. Reloads saved results and re-derives them from scratch rather than trusting the run that produced them. 25 checks. **Run this before the demo.**
`smoke_test.m`	Fast 10-point regression check of the whole package — toolbox availability, hover equilibrium, rotor signs, closed-loop tracking, `pidtune`, `fitnet`. Seconds to run; use it after any edit.

Outputs

File	Contents
`task1_model.mat`	`P, A, B, C, D, sys_full, sys_alt, G_alt`
`figures/01_task1_openloop_divergence.png`	2% thrust error diverging as t^2
`figures/02_task1_four_channels_pzmap.png`	pole-zero maps of all four channels

How to run

```
cd 'C:\Users\LENOVO\Downloads\HACKSIMU8\solution'
task1_statespace      % builds and verifies the model
verify_all            % independent 25-check verification
```

---

10. Presenting Task 1 (about 3 minutes)

1. **"6 DOF, 4 actuators — underactuated."** Show the state vector; point at `Ẍ = g·θ` and say the quadcopter must tilt to translate.
2. **"Thrust and torques come from ω^2."** Show the U1–U4 mapping; mention $\tau = r \times F$.
3. **"Here are the six nonlinear equations."** Point at `cos ϕ cos θ` in the altitude equation and flag that it drives Task 2.
4. **"Linearised about hover it decouples into four double integrators."** Show the table.
5. **"And here is the proof it's right."** `max|A − A_num| = 1.6e-12`, rank 12, zero hover residual.
6. **"All twelve poles at the origin — it can't fly open loop."** → hand to Task 2.

Questions and answers

"Why 12 states?" Six degrees of freedom, each second-order, so two states each: position and velocity.

"Why linearise if you have the nonlinear model?" Bode, root locus, margins and `pidtune` all assume linearity. We keep the nonlinear model for *simulation* and use the linear one for *design*, then verify the design back on the nonlinear model — Task 2's Method D does exactly that.

"Is the linear model valid at large angles?" No, it assumes small angles. That is precisely why the altitude controller uses feedback linearisation instead, which stays exact at any tilt (verified to 0.000262 m at 25.8°).

"You used Euler rates rather than body rates." Correct and deliberate — they coincide for small angles, and it is the formulation used by the cited reference. Here is the exact transformation between them.

"What did you neglect?" Aerodynamic drag on the airframe (coded but zero, because Table 1 gives no values — the paper's Eq. 6 has the terms but its Table 1 omits them too), blade flapping, motor dynamics, ground effect, and airframe flexibility (excluded by the stated assumptions).

"How do you know the model is right?" Twenty-five independent checks, including a finite-difference re-derivation of A and B that agrees to 1.6e-12. Run `verify_all` and watch.